Full Stack Engineer Application Project

Welcome to Kudwa's Full-Stack Developer Application Process! 🎉 We're eager to experience your expertise in backend integration and frontend development skills.

📂 Project Overview

In this full-stack challenge, you'll experience one of Kudwa's core technical challenges: **integrating diverse financial data sources into a unified, reliable source of truth**. You'll demonstrate your overall technical ability by integrating data sources on the backend and presenting the data simply and clearly on the frontend.

📌 Task Instructions

You have been provided two JSON API data sources representing financial data.

Your task is divided into two primary components:

🛠️ Backend Integration (Node.js & Express)

- **Framework:**
  - Implement your backend using Node.js and Express.
- **Data Integration & Schema Design:**
  - Create a unified database schema to store data from both provided JSON API sources.
  - Design and implement an ETL (Extract, Transform, Load) process to integrate and store data into the unified schema.

- **Migration Process:**
  - Include migration scripts for your schema setup and updates.
- **API Development:**
  - Develop simple API endpoints allowing the frontend to trigger data integration.

🎨 Frontend Display

- **Preferred Framework (optional):** React and Material UI
- **Screen 1: Structured Data Table**
  - Build a straightforward frontend table to display the integrated profit-and-loss report.
  - Include nested, expandable fields to demonstrate basic interactive functionality.
- **Screen 2 (Optional): Dashboard Visualization**
  - Optionally, build a simple dashboard highlighting key insights from the data. This step is entirely optional and considered a bonus.

⚙️ Frontend Controls

- Provide a clear button for users to trigger or refresh backend data integration.
- Provide simple feedback to indicate integration progress.

🧰 Bonus / Nice-to-Have Features

These additions are not mandatory, but they will make your application stand out:

- 📝 **Swagger / OpenAPI Documentation**

  Implement Swagger or OpenAPI to document your backend API endpoints. This helps demonstrate clarity and professionalism in API design.

- 🐳 **Docker Implementation**

  Use Docker to containerize the backend and/or frontend services. Provide a `docker-compose.yml` file if multiple services are used.

- 🧪 **Unit Testing**

  Write unit tests covering key backend API endpoints. Bonus points for using a testing framework like Jest or Mocha.

- 🚦 **API Error Handling & Response Standardization**

  Implement basic error handling for common failure scenarios and return standardized API response formats (e.g., consistent success/error structure).

📅 Timeline

- When you're ready, please notify us. You'll have **3 days** to complete and submit your solution.

🚀 Submission Guidelines

Submit your completed project including:

- Link to a live deployment of the application.
- Link to your GitHub repository, clearly documenting your commits, schema design, ETL scripts, and frontend structure.

🌟 Evaluation Criteria

- **Data Integration Quality:** Effectiveness and clarity of ETL process and schema design.
- **Clarity and UX:** Simple, intuitive interfaces.
- **Quality of Code:** Clean, maintainable, clearly documented code.
- **Functionality:** Completeness in handling and displaying integrated data.

We're excited to review your solution! 🚀💻

💽 Data Required for Exercise

- You will be required to use the below dataset

data_set_1.json 1.1 MB

data_set_2.json 485.8 KB